

Duplicate Merge Control

Instruction Manual

Contents

1. Introduction	2
2. Prerequisites.....	2
3. Solution Import	3
4. Dataverse Configuration	4
5. Model-Driven App Setup	5
6 Adapting to Different Tables	7
6.1 Polymorphic Lookup Changes.....	7
6.2 PCF Codebase Changes	8

1. Introduction

The Duplicate Merge Control is a Power Apps Component Framework (PCF) component for Microsoft Dynamics 365 that allows users to compare and merge duplicate records directly inside a model-driven app. It is designed to work together with a duplicate detection plugin, which identifies duplicate records and creates groups based on them.

When a duplicate group is opened in the model-driven app, the control displays a grid containing all linked duplicate records. These referenced records can belong to any standard or custom table. The records are shown side by side, allowing the user to select the values they want to keep. After all fields have been selected, the user can start the merge process. The primary record is kept and updated with the selected values, while all duplicate records are deactivated.

The purpose of this control is to replace the default Dynamics duplicate merge interface, which only supports Accounts and Contacts. This control allows duplicate records from any table to be merged.

2. Prerequisites

Before beginning the setup, the following must be in place in the target environment.

Requirement	Details
Dynamics Environment	Any active environment with model-driven app capabilities
Sys. Admin role	Required for solution imports and configuration
Power Apps CLI	Needed for modifying the PCF code
Managed solution file	The project is contained in the solution file

For Extending the solution to other tables:

Requirement	Details
PCF Codebase	The zip file containing the PCF code
XRM Toolbox	Required to add new tables to the reference table

3. Solution Import

1. Open make.powerapps.com and select the target environment.
2. Open the **Solutions** page from the left navigation panel.
3. Click Import solution in the top toolbar. Select the managed solution .zip file and click Next.
4. Review the solution details. No additional connection references or environment variables are required. Click Import.
5. After the import finishes, open the solution and verify that the following components exist
 - a. Duplicate Group | Table
 - b. Duplicate Record Reference | Table
 - c. Godiz.Controls.DuplicateMergeGrid | custom component

If the import fails, check the following:

Issue	Likely cause	Resolution
dependency error	required component is missing in target env.	Ensure the target is a valid environment which supports model-driven apps
permission error	Insufficient privileges	Verify that the user is a system admin
components missing	Corrupted or incomplete solution file	Re-download the solution file and re-import
control not visible in "more components"	Solution imported but not published	Navigate to Solutions, select the solution, and click Publish all customisations

If the steps above fail, contact the solution owner at siahspilker@gmail.com

4. Dataverse Configuration

The solution includes two custom tables that act as the data foundation for the control. These tables are imported automatically with the solution and require no additional column configuration.

`jsp_duplicategroup`

Acts as the container record where the PCF control is embedded. Each record represents one merge job and is referenced by multiple duplicate record references.

`jsp_duplicaterecordreference`

Represents a row in the merge grid. Each record contains a polymorphic lookup to the referenced table.

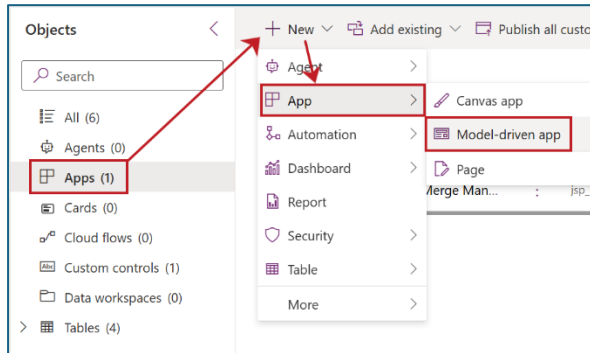
To manually test the control data:

1. Create a `jsp_duplicategroup` record.
2. Set Target Table to the logical name of the entity being merged, such as account or contact.
3. Create one `jsp_duplicaterecordreference` row for each duplicate record.
4. Set the Referenced Record polymorphic lookup to the matching Account or Contact.
5. Set Is Primary to Yes on the reference row that should become the merge target.
6. Set the Primary Record lookup on the group to the reference row marked as primary.
7. Ensure that between two and five reference rows are linked, as the control does not support fewer than two or more than five records.

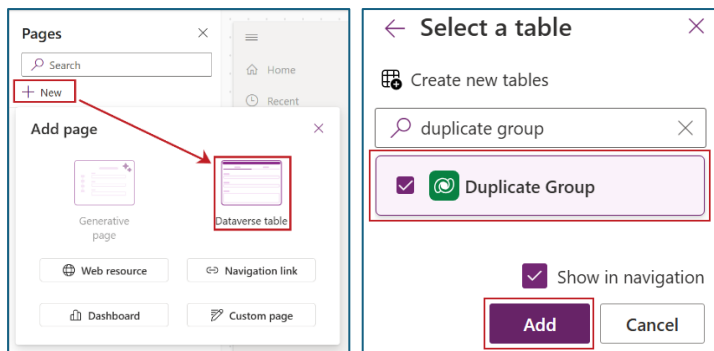
5. Model-Driven App Setup

To use the Duplicate Merge component, a model-driven app must first be created in the target environment.

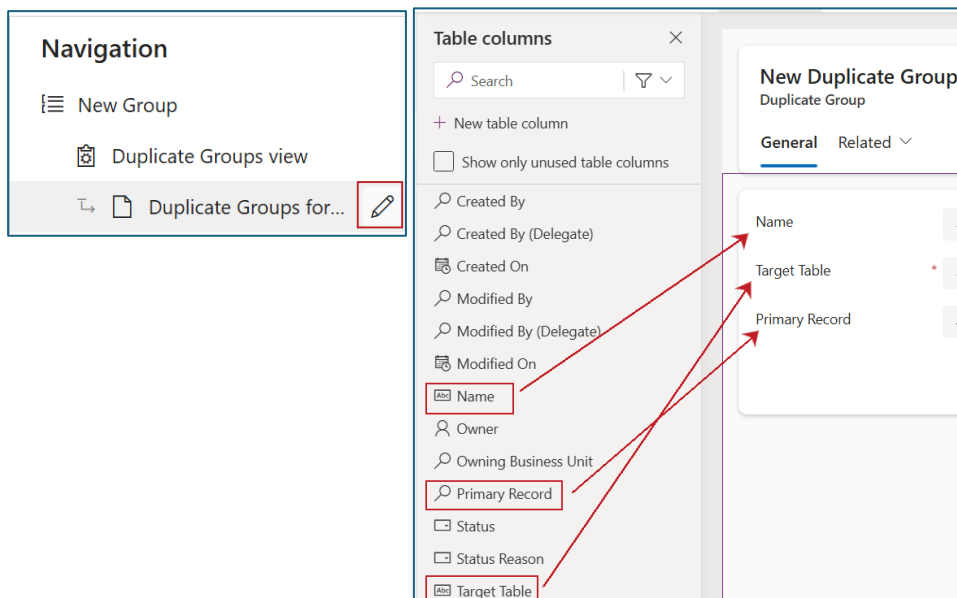
1. To create a model-driven app navigate to Apps, New app, Model-driven app. Give the app a name and click Create.



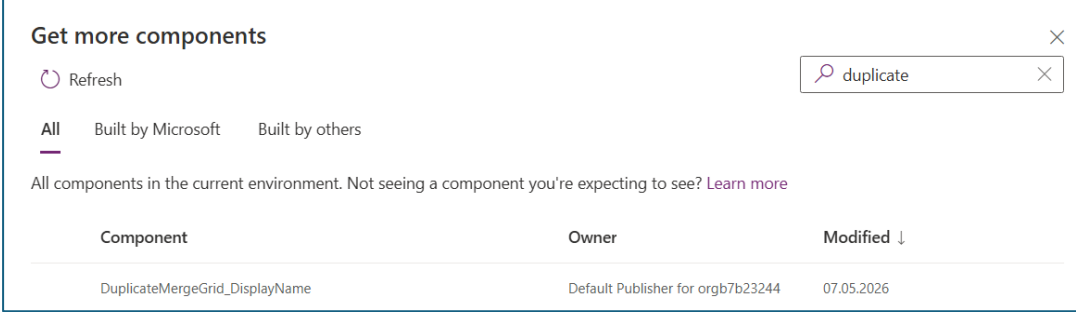
2. To add the Duplicate Group table in the app designer click Add page and select Dataverse table. Search for and select Duplicate Group. Click Add.



3. You can then open the Main form. Ensure that the columns Name, Target table and primary record are included. These columns can be dragged and dropped onto the form to include them. While not required for the PCF they are helpful information.



4. Add the PCF control to the form on the main form of Duplicate Group. With the main form open in edit mode, click Components in the right panel, then click Get more components. Search for DuplicateMergeGrid and add it to the app.



Get more components

Refresh

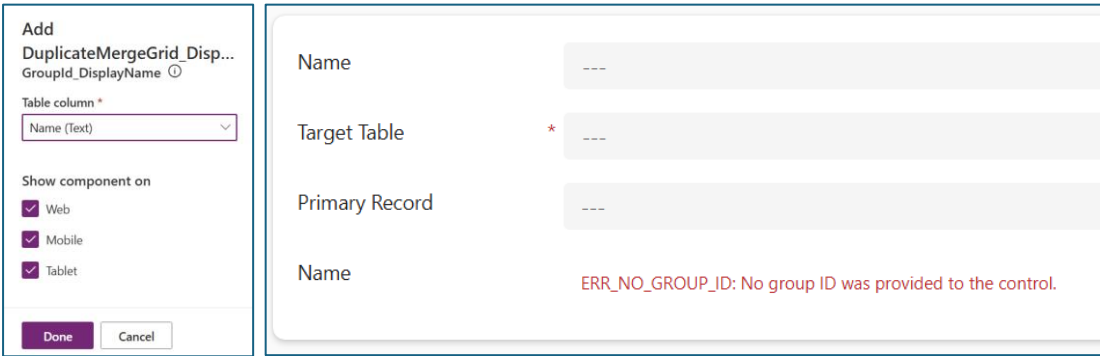
duplicate

All Built by Microsoft Built by others

All components in the current environment. Not seeing a component you're expecting to see? [Learn more](#)

Component	Owner	Modified ↓
DuplicateMergeGrid_DisplayName	Default Publisher for orgb7b23244	07.05.2026

Drag the component onto the form. A value must be selected during setup, although it has no effect on the PCF itself. At this stage, the component may display the error message: "No group ID was provided..." This is expected when the form is still empty.



Add
DuplicateMergeGrid_DisplayName
GroupID_DisplayName ⓘ

Table column *

Name (Text)

Show component on

☒ Web

☒ Mobile

☒ Tablet

Done Cancel

Name

Target Table *

Primary Record

Name

ERR_NO_GROUP_ID: No group ID was provided to the control.

5. Click Save then Publish on the form. Return to the app designer and click Publish on the app.

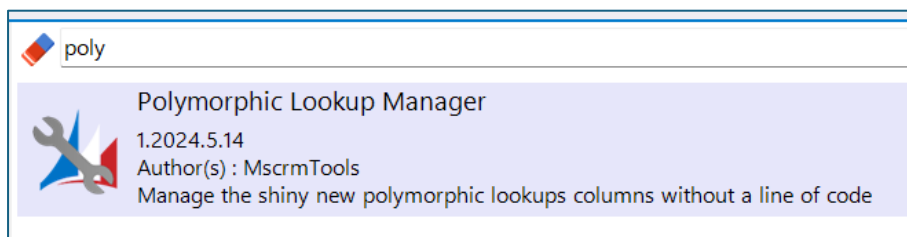
6 Adapting to Different Tables

DuplicateMergeGrid supports the Account and Contact tables by default. To add support for another table, such as Lead, two changes are required:

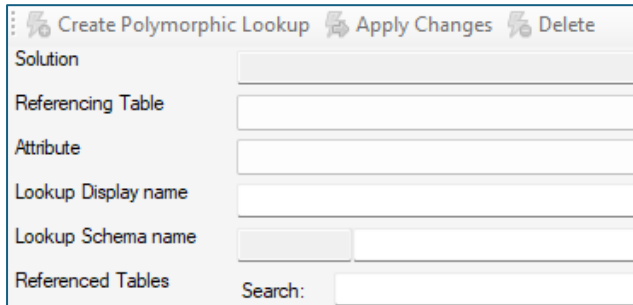
1. Update the polymorphic lookup in Dataverse.
2. Update the PCF codebase.

6.1 Polymorphic Lookup Changes

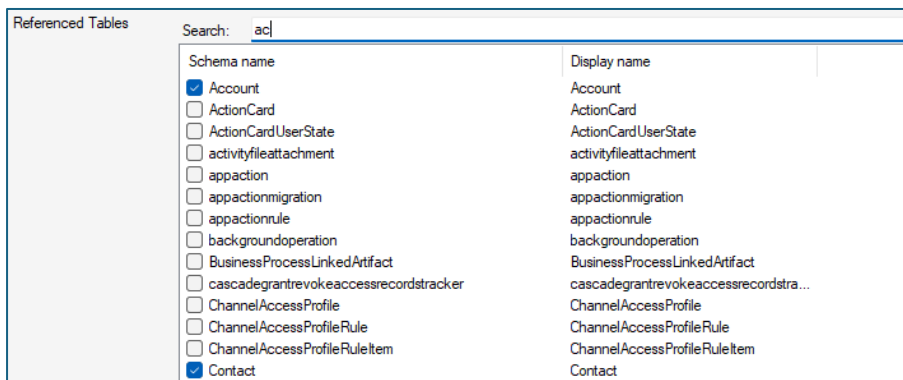
Although modifying polymorphic lookups directly in Dynamics is difficult, this can be done easily using the XRM Toolbox tool called Polymorphic Lookup Manager.



Connect to your environment using the connection menu, you will then be greeted with this menu:



First, connect to your solution with the model-driven app, select the referencing table `jsp_duplicaterecordreference`, then the Attribute `jsp_referencedrecordid` and lastly select or remove the referenced tables you require in the respective menu.



Schema name	Display name
☒ Account	Account
☐ ActionCard	ActionCard
☐ ActionCardUserState	ActionCardUserState
☐ activityfileattachment	activityfileattachment
☐ appaction	appaction
☐ appactionmigration	appactionmigration
☐ appactionrule	appactionrule
☐ backgroundoperation	backgroundoperation
☐ BusinessProcessLinkedArtifact	BusinessProcessLinkedArtifact
☐ cascadegrantrevokeaccessrecordtracker	cascadegrantrevokeaccessrecordstra...
☐ ChannelAccessProfile	ChannelAccessProfile
☐ ChannelAccessProfileRule	ChannelAccessProfileRule
☐ ChannelAccessProfileRuleItem	ChannelAccessProfileRuleItem
☒ Contact	Contact

6.2 PCF Codebase Changes

All changes are confined to a single file: `utils/WebApiService.ts`.

1. Add a link-entity block in the FetchXML query (inside `fetchDuplicateGroup`)

Add a new outer join for the target table alongside the existing `acc.` and `con.` blocks. You can also remove existing joins that are not required. Use a short alias and include the columns you want the user to compare and merge:

```
<link-entity name="lead" from="leadid" to="jsp_referencedrecordid" link-  
type="outer" alias="lead">  
  <attribute name="leadid" />  
  <attribute name="fullname" />  
  <attribute name="emailaddress1" />  
  <attribute name="companyname" />  
</link-entity>
```

2. Add a case in `resolveRecordId`

Return the primary key from the new alias:

```
if (targetTable === "lead") return entity["lead.leadid"] as string ?? "";
```

3. Add a case in `resolveRecordName`

Return the display name field:

```
if (targetTable === "lead") return entity["lead.fullname"] as string ??  
"Unknown";
```

4. Add a case in `resolveFields`

Map each column to an `IFieldValue` entry with the appropriate type (text, choice, or lookup):

```
if (targetTable === "lead") {  
  return {  
    fullname: { raw: entity["lead.fullname"], formatted:  
entity["lead.fullname"] as string ?? " ", type: "text" },  
    emailaddress1: { raw: entity["lead.emailaddress1"], formatted:  
entity["lead.emailaddress1"] as string ?? " ", type: "text" },  
    companyname: { raw: entity["lead.companyname"], formatted:  
entity["lead.companyname"] as string ?? " ", type: "text" },  
  };  
}
```


For lookup fields, include the `lookupName` and `lookupEntityType` properties and set type: `"lookup"`.

For choice/optionset fields, read the formatted value from the OData annotation and set type: `"choice"`.

No changes are needed to the React components, styles, handlers, or merge logic as they operate generically on whatever fields `resolveFields` returns.

Verification

After making the changes, rebuild and push the control using the `pac PCF push` command. Create a duplicate group with `jsp_targettable` set to the new table's logical name, link at least two record references, and open the group form to confirm the grid loads and the merge executes correctly.